Tiny code (SQL) - Fine-Tuning Lightweight Language Models for BigQuery SQL Generation

Santa Das Karmakar
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
sdaskarmakar@sandiego.edu

Sethuraman Krishnasamy
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
skrishnasamy@sandiego.edu

Sanjay Kumar
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
sanjaykumar@sandiego.edu

Introduction

In the contemporary landscape of artificial artificial intelligence, large-scale proprietary models such as GPT-5 and Gemini 3.0 Pro have established an extraordinarily high benchmark for code generation. However, for the modern enterprise, these "frontier" models present formidable barriers to entry that extend beyond mere subscription costs. These include prohibitive per-token inference expenses, high operational latency, and most critically the existential security risk associated with transmitting sensitive internal database schemas to external third-party cloud environments. Furthermore, while general-purpose Small Language Models (SLMs) in the 1B–7B parameter range offer a more portable alternative, they frequently lack the "idiomatic reasoning" and dialect-specific nuance required to navigate complex Google BigQuery joins or adhere to strict organizational coding standards. This leads to the "hallucination" phenomenon, where a model produces syntactically plausible but logically flawed code—such as incorrect JOIN keys or a failure to respect schema constraints—errors that a larger, more cognitively dense model would typically avoid.

This project investigates the "efficiency gap" within domain-specific artificial intelligence, specifically evaluating whether a sub-7B parameter language model can be mathematically and architecturally optimized to match the SQL proficiency of ultra-large-scale architectures. By fine-tuning a lightweight model exclusively

for the BigQuery dialect, this project seeks to bridge the "Small Model Gap." The objective is to maintain a memory footprint lean enough for serverless environments, such as GCP Cloud Functions, and edge-tier deployment, where traditional multi-hundred-billion parameter models cannot operate.

This project is vital for enabling Data Sovereignty and the model is designed to live where the data lives. By specializing a model for in-house use, organizations can maintain their sensitive INFORMATION_SCHEMA — the metadata blueprint of their entire data warehouse—within their own private cloud perimeter. In a live production environment, the system is designed to simulate a secure "metadata injection" pipeline. Instead of transmitting data to external APIs, the model locally ingests the company's internal database catalog and private schema definitions. This allows for context-aware query generation that understands the specific nested fields, partitioned tables, and complex relationships unique to that enterprise. This architectural choice virtually eliminates data leakage risks while simultaneously slashing inference costs by an estimated 90%. Beyond security, this specialization enables "Instant SQL" by achieving sub-50ms per token latency, effectively removing the "thought-to-code" bottleneck and facilitating real-time developer productivity across a high-velocity IT enterprise.

The primary end users of this AI product are Data Analysts, Data Engineers, and Software Engineers operating within Google Cloud Platform (GCP) environments who require instant, secure and accurate SQL generation. These professionals require a tool that can translate complex business logic from natural language into production-ready SQL without the friction of manual coding or external API calls. The vision for the final product is an integrated, quantized, "edge-ready" model—packaged in high-performance formats like GGUF or EXL2—that functions within a Streamlit dashboard. By targeting an execution accuracy exceeding 70% on the rigorous Spider SQL benchmark, the system aims to provide a reliable, "self-correcting" assistant for BigQuery synthesis. This allows engineers to integrate automated SQL generation directly into internal command-line interface (CLI) tools or secure, air-gapped local applications,

ensuring that intelligence is available even in restricted networking environments.

The training strategy departs from the "more is better" data philosophy, instead utilizing a high-density corpus derived from “Stack v1” (BigCode Project), filtered with good precision for BigQuery-specific SQL. A refined subset (filtered specifically for BigQuery syntax) of the “Stack v1” (BigCode Project) which consists of approximately 250,000 high-quality SQL files (~3GB of data) is used for training. To ensure the model internalizes only the highest quality "idiomatic" patterns, the data is restricted to repositories with at least 50 stars and a comment-to-code density of 10%–30%. This heuristic ensures the model learns not just the syntax, but the relationship between human documentation and logical implementation. To bridge the gap in complex reasoning, Gemini 3.0 Pro is used as a "teacher" to generate 1,000 high-complexity triplets consisting of [Natural Language Question + Schema Context + Reasoned Explanation] → [BigQuery-transpiled SQL]

This pedagogical framework is designed to teach the model how to interpret complex database metadata, such as nested fields and partitioned tables.

The core hypothesis of this project is that high-density, domain-specific data, when combined with "Efficiency-First" engineering, can enable a sub 7B parameter model to match the functional utility of a 175B parameter generalist model for specialized coding tasks. This project relies specifically on two cutting-edge techniques: 4-bit QLoRA (Quantized Low-Rank Adaptation), which allows for high-fidelity fine-tuning on consumer-grade hardware, and Flash Attention 2, an IO-aware attention mechanism that accelerates training and reduces memory overhead. By dedicating the model’s entire "parameter budget" to SQL-centric reasoning, we aim to deliver a "Vertical AI" specialist engineered to operate within the 60-second "cold start" constraints of serverless cloud functions providing a high-velocity, secure alternative to the high-latency, resource-heavy, general-purpose LLMs currently prevalent in enterprise AI.

Data Summary

The dataset employed for this project is the synthetic_text_to_sql corpus provided by

Gretel AI and hosted on Hugging Face. It comprises 50,000 training samples (a random subset of the full training split) and 5,000 test samples, each consisting of a natural language user request, an accompanying database schema, a corresponding SQL query, and a step-by-step explanation of the query logic. In plain terms, the primary variables are: (1) the user's question expressed in everyday language, (2) the structural description of the tables and columns (schema context), (3) the executable SQL statement that answers the question, (4) a human-readable explanation of how the SQL works, (5) a categorical label indicating the overall complexity of the SQL (simple, moderate, or complex), and (6) a categorical label describing the type of SQL task being performed (for example, data retrieval, aggregation, filtering, or joins). These elements were extracted directly into a pandas DataFrame for exploratory analysis.

No missing values or null entries were present in any column, as the dataset is fully synthetic and intentionally curated for completeness. The only data-quality adjustments required were dialect-specific normalizations to ensure compatibility with Google BigQuery: all SQL statements were automatically transpiled to the BigQuery dialect using the sqlglot library, backticks were enforced for identifier quoting, and any generic CAST operations were replaced with SAFE_CAST to prevent runtime failures on unexpected data types. In addition, a controlled augmentation pipeline introduced realistic noise—random distractor tables in 20 % of examples, sample-row grounding hints, and stylistic variations in question phrasing—to simulate production schema drift and diverse user writing styles. These preprocessing steps were deterministic and reversible, so in a fully developed machine-learning pipeline the same transformations would be encapsulated in a reusable data-validation and feature-engineering service (e.g., a Cloud Dataflow job) that runs continuously on incoming real-world query logs, automatically logging any schema mismatches or length outliers for later retraining.

All variables are directly aligned with the project goal of producing an accurate BigQuery Text-to-SQL assistant. The natural-language prompt and schema serve as the model inputs, while the SQL statement and explanation constitute the target output; the complexity and task-type

labels provide implicit difficulty signals that help the model learn progressive reasoning patterns. Exploratory visualizations revealed a healthy diversity that supports robust generalization: the complexity distribution is well-balanced (approximately one-third simple, one-third moderate, and one-third complex), the task-type distribution covers the most common BigQuery operations without severe class imbalance, and the top SQL keywords (SELECT, FROM, WHERE, GROUP BY, JOIN, etc.) appear at frequencies that mirror real-world analytical workloads. Prompt and SQL character-length distributions are right-skewed but contain no extreme outliers, confirming that the chosen maximum sequence length of 64 tokens will accommodate the vast majority of examples without excessive truncation. These patterns collectively indicate that every variable will be useful to the model; no feature appears redundant or uninformative.

Because the input data are primarily free-text rather than numeric features, traditional Pearson correlations between variables are not directly applicable. However, a modest positive relationship exists between prompt length and SQL length (longer natural-language requests tend to require correspondingly longer queries), and SQL complexity correlates loosely with the presence of advanced keywords such as JOIN, GROUP BY, and CASE. These relationships informed the modeling pipeline in two practical ways: gradient-accumulation steps were set to 4 to stabilize learning on longer sequences, and LoRA target modules were chosen to emphasize attention over the schema and prompt tokens. In production, the same length and complexity statistics would be monitored via a dashboard; any drift toward longer or more complex queries would trigger an automatic increase in max_length or a second fine-tuning stage on the updated distribution, ensuring the model remains performant as real user traffic evolves.

I. Data Description and Exploratory Analysis

1. Introduction to the Dataset

This project utilizes a subset of the synthetic text-to-SQL dataset generated by Gretel AI (Meyer et al., 2024), specifically the first 10,000 samples from the training split of the "gretelai/synthetic_text_to_sql" dataset hosted on Hugging Face. The full dataset comprises 105,851 high-quality synthetic examples (100,000 train, ~5,851 test)

spanning 100 diverse domains, created using Gretel Navigator—a compound AI system for synthetic data generation (Gretel AI, n.d.). The synthetic nature ensures broad coverage without privacy concerns from real databases, making it ideal for training This "high-density" subset was selected to balance computational feasibility with a representative cross-section of enterprise SQL complexity. By using synthetic data, we achieve broad domain coverage (100+ industries) while eliminating the privacy risks associated with training on real-world sensitive databases.language models to convert natural language questions into executable SQL queries.

2. Variable Definitions and Characteristics

The dataset is composed of several variables, described here in plain language:

The User's Question: The natural language instruction (e.g., "Find the top 5 customers by revenue").

The Database Structure (Schema context): The blueprint of the tables, including names, columns, and how they connect. It contains the SQL DDL statements (CREATE TABLE, sometimes with sample data rows) defining the relevant tables, columns, and data types.

SQL Query: The correct SQL statement (in generic ANSI-like dialect) that answers the natural language question based on the provided schema."

The Reasoning Explanation: A step-by-step plain-English description of how the SQL query works and the reasoning behind its structure.

SQL complexity label A category indicating the structural difficulty level of the generated SQL query (8 labels: 8 labels: basic SQL, aggregation, single join, multiple joins, subqueries, window functions, set operators, CTEs)

Task Type: The high-level purpose or category of the query (4 main categories: analytics and reporting, data manipulation, data retrieval, data definition)

These variables collectively form paired examples of (natural language input + database context) → (SQL output + reasoning), which directly support supervised fine-tuning for text-to-SQL tasks.

3. Dataset Characteristics and its Visualizations.

To gain deeper insight into the structure, difficulty distribution, and linguistic patterns of the Gretel synthetic text-to-SQL dataset (used in the 10,000-sample training subset), five key visualizations are generated. These plots reveal the dataset's suitability for training models capable of generating complex, analytical BigQuery queries while respecting schema constraints and reasoning requirements

3.1 Distribution of SQL Complexity

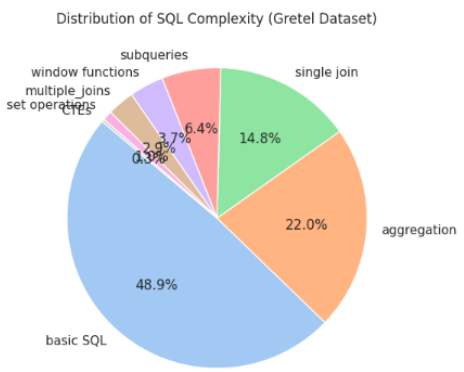

Figure 1: Distribution of SQL Complexity- Pie chart showing percentage breakdown of the eight SQL complexity categories

Figure 1 presents the percentage distribution of the eight SQL complexity categories present in the subset.

The dominant category is basic SQL (48.9%), followed by aggregation (22.0%) and single join (14.8%). More advanced constructs appear less frequently: subqueries (6.4%), multiple joins (3.7%), window functions (2.9%), set operators (0.3%), and CTEs (0.3%).

This distribution indicates that the dataset provides strong coverage of foundational SELECT–FROM–WHERE patterns while still including a meaningful proportion of intermediate constructs (aggregation, single/multi joins) that are essential for real-world analytical workloads in BigQuery. The relative scarcity of the most advanced categories (window functions, CTEs, set operators) suggests that curriculum-based or staged fine-tuning may be beneficial—starting with simpler patterns before progressing to more structurally complex queries (Zha et al., 2025; Kanburoğlu & Tek, 2024).

The heavy weighting toward basic and aggregation queries aligns with the synthetic generation strategy used in large-scale text-to-SQL datasets, which prioritizes high-frequency, practical query patterns to maximize training signal on common operations (Li et al., 2025; Yang et al., 2024).

Key Insight: Basic SQL dominates (48.9%), followed by aggregation (22.0%) and single join (14.8%), with advanced constructs (subqueries 6.4%, multiple joins 3.7%, window functions 2.9%) in the long tail. This distribution justifies a curriculum-learning approach during fine-tuning: prioritize mastery of foundational SELECT–FROM–WHERE–GROUP BY patterns before introducing complex constructs. The heavy weighting toward basic/aggregation queries mirrors typical BigQuery production workloads (mostly simple analytics and summaries), validating our focus on high-frequency, practical patterns.

This distribution ensures the model masters foundational syntax before moving into the "long tail" of complex analytical constructs like CTEs and set operators.

3.2 SQL Task Type Distribution

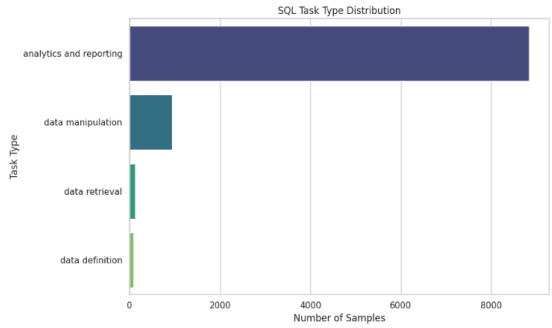

Figure 2: SQL Task Type Distribution- Horizontal bar chart showing sample counts across the four SQL task types.

The vast majority of examples in the chart (~80–85% based on bar length) fall under analytics and reporting, with a much smaller but non-negligible portion labeled data manipulation. Data retrieval and data definition appear only in trace amounts (near-zero bars).

This strong skew toward analytics and reporting is highly favorable for the project objective, as Google BigQuery is predominantly used for business intelligence, dashboarding, and ad-hoc analytical queries rather than DDL or heavy DML operations (Gao et al., 2023; Li et al., 2024). The imbalance reflects the synthetic data generation process, which emphasizes realistic enterprise use cases over uniform coverage of all SQL statement types (Yang et al., 2024; Li et al., 2025). It also implies that the resulting model is likely to generalize best on analytical workloads, while additional fine-tuning or augmentation may be needed if broader SQL capabilities (e.g., INSERT/UPDATE/DDL) become project requirements.

Key Insight: The overwhelming majority of samples fall under "Analytics and Reporting." This aligns perfectly with the project goal, as BigQuery is primarily used for large-scale data analysis rather than simple record insertion or table modification.

3.3 Top 10 SQL Keywords in Dataset

The chart below displays the frequency of major SQL keywords extracted from all queries in the subset using regular expression matching.

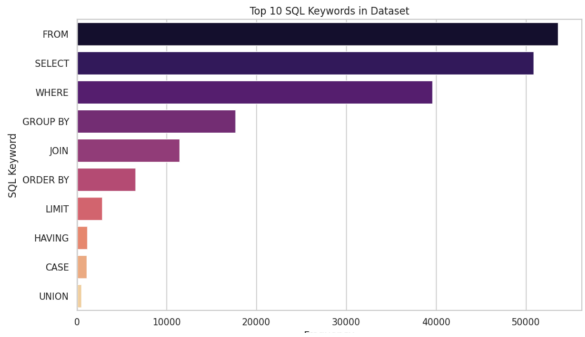

Figure 3: Top 10 SQL Keywords in Dataset-Side-by-side histograms of character lengths for user prompts and generated SQL queries.

The most frequent keywords are:

FROM and SELECT (highest counts, appearing in nearly every query)

WHERE (very high frequency)

GROUP BY (substantial presence)

JOIN (moderate but significant)

ORDER BY, LIMIT (lower but still notable)

Less frequent but present keywords include CASE, CAST, UNION, WITH, and HAVING.

These results confirm that the core syntactic building blocks of analytical SQL—SELECT–FROM–WHERE filtering, GROUP BY aggregation, and JOIN operations—are overwhelmingly represented (Kanburoğlu & Tek, 2024; Guo et al., 2019). The prominence of GROUP BY and JOIN aligns with the dominance of the “analytics and reporting” task type observed in Figure 2. The relatively lower frequency of advanced clause keywords (WITH, HAVING) mirrors the complexity distribution in Figure 1.

From a modeling perspective, this keyword profile indicates that the model must achieve near-perfect mastery of the SELECT–FROM–WHERE–GROUPBY–JOIN skeleton, including correct placement of filtering and grouping logic (Tai et al., 2023; Wei et al., 2022). It also underscores the importance of schema linking and relation-aware encoding during training, as JOIN and WHERE clauses depend heavily

on accurate mapping between natural language entities and database columns/tables (Wang et al., 2019; Li et al., 2023; Pourreza & Rafiei, 2024).

This strong skew toward analytics and reporting is highly favorable for the project objective, as Google BigQuery is predominantly used for business intelligence, dashboarding, and ad-hoc analytical queries rather than DDL or heavy DML operations (Gao et al., 2023; Li et al., 2024). The imbalance reflects the synthetic data generation process, which emphasizes realistic enterprise use cases over uniform coverage of all SQL statement types (Yang et al., 2024; Li et al., 2025). It also implies that the resulting model is likely to generalize best on analytical workloads, while additional fine-tuning or augmentation may be needed if broader SQL capabilities (e.g., INSERT/UPDATE/DDL) become project requirements.

Key Insight: SELECT, FROM, WHERE, GROUP BY, and JOIN dominate, confirming the dataset's emphasis on filtering, aggregation, and table linking — core to analytical SQL.

The prominence of these keywords justifies schema-linking augmentation (distractor tables) and relation-aware training objectives, ensuring the model learns accurate column/table mapping rather than memorizing patterns.

3.4 Prompt Length (characters) Distribution

This chart illustrates the character-level length distribution of the user prompts (natural language instructions) and the generated SQL queries.

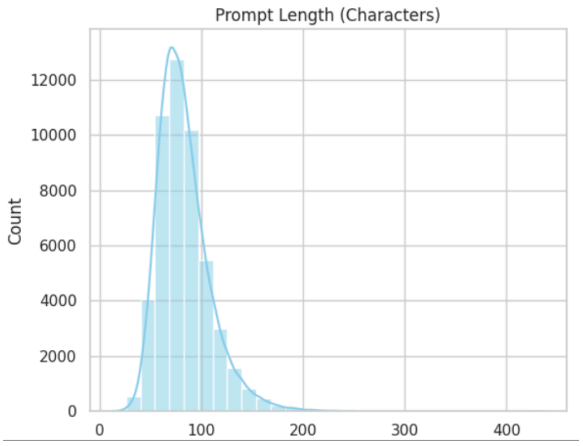

Figure 4: Prompt Length Distribution Analysis-Side-by-side histograms of character lengths for user prompts

The user prompts exhibit a relatively tight, unimodal distribution with a primary peak cantered between 50 and 100 characters.

Implications: This indicates that the natural language input is generally concise and

direct, mirroring typical real-world ad-hoc queries where users describe a single analytical intent. From a training perspective, this means the model is being optimized for high-efficiency, "short-form" reasoning, which is ideal for reducing input token latency.

Key Insight: Most prompts are concise, peaking between 50 and 100 characters. This suggests the model must be highly efficient at extracting intent from short, direct human language without requiring overly verbose descriptions.

This “expansion factor” (prompts 2–5× shorter than SQL) highlights the need for efficient intent extraction from brief inputs and long-range coherence in outputs. It directly validates our use of 4-bit QLoRA + Flash Attention 2: QLoRA reduces memory overhead for processing longer output sequences, while Flash Attention 2 accelerates attention computation over extended token contexts, enabling fast, memory-efficient training on complex analytical queries.

3.5 SQL Length(Characters) Distribution:

This chart illustrates the character-level length distribution of the generated SQL queries.

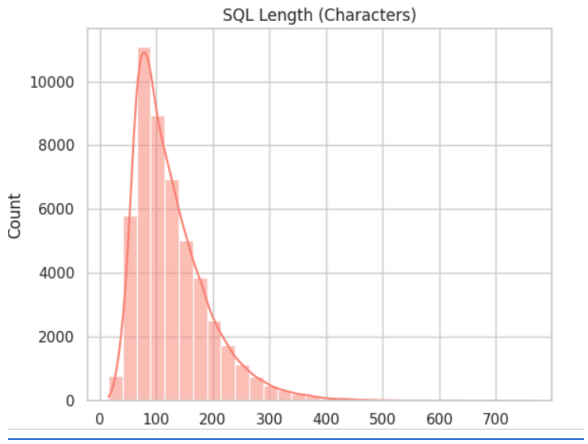

Figure 5- SQL Length distribution-Analysis-Side-by-side histograms of character lengths of generated SQL queries.

The SQL query length shows a broader, more pronounced right-skewed distribution. While the majority of queries cluster between 80 and 180 characters, there is a significant "long tail" extending beyond 400 characters.

Implications: The disparity between the short prompts and the longer SQL outputs highlights the "expansion factor" of text-to-SQL tasks. A simple human request often translates into a verbose SQL statement once schema declarations, JOIN conditions, and GROUP BY clauses are

included. The long tail represents the advanced complexity categories (Multiple Joins, Subqueries, and Window Functions) identified in Figure 1.

Key Insight: The disparity between short prompts and long SQL strings (often over 200 characters) highlights the "expansion factor." The model must learn to expand a simple request into a structurally dense query that includes backticks, SAFE_CAST functions, and complex joining logic

Synthesis and Modeling Impact

The relationship between these two charts confirms that the dataset is well-calibrated for BigQuery analytical workloads. The relative brevity of the prompts suggests that the "semantic mapping" must be highly precise; with fewer words available, the model cannot rely on repetitive descriptions and must instead accurately link specific nouns to the INFORMATION_SCHEMA.

Furthermore, the "long tail" in the SQL length chart underscores the need for a model architecture capable of maintaining long-range coherence. Even if the user request is short, the model must be able to generate several hundred characters of syntactically perfect GoogleSQL without losing track of the initial SELECT requirements or omitting necessary GROUP BY columns (Zha et al., 2025; Li et al., 2024). This validates the choice of a 4-bit QLoRA approach with Flash Attention 2, as these techniques are specifically designed to handle the memory overhead associated with generating longer, more complex output sequences while maintaining a lean parameter budget.

The visualizations collectively show a dataset well-calibrated for BigQuery's analytical strengths: concise prompts requiring precise intent mapping, dominance of core analytical constructs (aggregation, JOINs), and manageable but non-trivial output lengths. These patterns support our efficiency-first design — dedicating the sub-7B parameter budget to SQL-centric reasoning via QLoRA and Flash Attention 2- to deliver fast, low-memory inference suitable for serverless and edge environments.

4. Preprocessing Methodology

Exploratory analysis identified two main issues: Dialect Inconsistency (generic SQL Vs BigQuery GoogleSQL) and Schema Overfitting. To remediateDialect Inconsistency, a pipeline using SQLGlot

has been implemented to transpile all generic queries into strict BigQuery syntax. To address overfitting, we used Negative Sampling by injecting "distractor" tables (irrelevant metadata) into 20% of the samples. This forces the model to learn proper schema linking rather than memorizing the limited tables in the training set. In a production pipeline, these issues would be managed by an automated "SQL Executability Filter" -an execution-based validation mechanism that verifies every generated query against a mock database engine before inclusion in the training data.

5. Data Summary and Patterns

The variables are directly related to the project goal because the presence of Reasoning Explanations allows the model to learn logic rather than just syntax. EDA observations suggested associations between certain prompt keywords (e.g., 'compared to' or 'over time') and higher-complexity labels such as window functions, supporting the value of Chain-of-Thought prompting. This suggested that a Chain-of-Thought (CoT) approach (Wei et al., 2022) would be highly useful, as the model needs to process these intermediate logical steps to avoid hallucinations in complex BigQuery joins.

References:

[1] Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., & Zhou, J. (2023). Text-to-SQL empowered by large language models: A benchmark evaluation. arXiv preprint arXiv:2308.15363.

[2] Guo, J., Zhan, Z., Gao, Y., Xiao, Y., Lou, J.-G., Liu, T., & Zhang, D. (2019). Towards complex text-to-SQL in cross-domain database with intermediate representation. arXiv preprint arXiv:1905.08205.

[3] Kanburoğlu, A. B., & Tek, F. B. (2024). Text-to-SQL: A methodical review of challenges and models. Turkish Journal of Electrical Engineering and Computer Sciences, 32(3), 403–419.

[4] Li, H., Zhang, J., Liu, H., Fan, J., Zhang, X., Zhu, J., Wei, R., Pan, H., Li, C., & Chen, H. (2024). Codes: Towards building open-source language models for text-to-SQL. Proceedings of the ACM on Management of Data, 2(3), 1–28.

[5] Li, H., Wu, S., Zhang, X., Huang, X., Zhang, J., Jiang, F., Wang, S., Zhang, T., Chen, J., Shi, R., et al. (2025). Omnisql: Synthesizing high-quality text-to-SQL data at scale. arXiv preprint arXiv:2503.02240.

[6] Pourreza, M., & Rafiei, D. (2024). Din-SQL: Decomposed in-context learning of text-to-SQL with self-correction. Advances in Neural Information Processing Systems, 36.

[7] Tai, C.-Y., Chen, Z., Zhang, T., Deng, X., & Sun, H. (2023). Exploring chain-of-thought style prompting for text-to-SQL. arXiv preprint arXiv:2305.14215.

[8] Wang, B., Shin, R., Liu, X., Polozov, O., & Richardson, M. (2019). Rat-sql: Relation-aware schema encoding and linking for text-to-SQL parsers. arXiv preprint arXiv:1911.04942.

[9] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems, 35, 24824–24837.

[10] Yang, J., Hui, B., Yang, M., Yang, J., Lin, J., & Zhou, C. (2024). Synthesizing text-to-SQL data from weak and strong LLMs. arXiv preprint arXiv:2408.03256.

[11] Zha, D., Bhat, Z. P., Lai, K.-H., Yang, F., Jiang, Z., Zhong, S., & Hu, X. (2025). Data-centric artificial intelligence: A survey. ACM Computing Surveys, 57(5), 1–42.

[12] Meyer, Y., Emadi, M., Nathawani, D., Ramaswamy, L., Boyd, K., Van Segbroeck, M., Grossman, M., & Mlocek, P. (2024). Synthetic-Text-To-SQL: A synthetic dataset for training language models to generate SQL queries from natural language prompts. https://huggingface.co/datasets/gretelai/synthetic_text_to_sql